

Технологии программирования



Во имя человечества

Тип В номер 2

Typedef

- Определяет синоним (alias) для существующего типа.
- Основное использование - для сокращения написания длинных типов,

```
typedef unsigned int      uint32;  
typedef std::vector<uint32> ivector32;  
ключ   | исходны тип   | алиас
```

- Для придания коду собственного, уникального вида.....

```
typedef unsigned int DWORD;  
typedef int         INT;  
typedef short       WCHAR; // Большое тебе спасибо... Microsoft >:(
```

- Для взятия указателя на функцию:
 - Стандартных типов кроме auto для получения указателя на функцию

```
typedef Object(*functionPointerType)(const Parameter*, Result*);  
ключ   | retval | имя типа функции | список параметров
```

- Если возвращаемое значение и список параметров у функции и у типа отличаются, то функцию к этому типу привести нельзя

```
using u32 = unsigned int; // то же что и typedef unsigned int u32
```



Близнецы



Классический C интерфейс

- Для каждого типа параметров создается своя функция, со своим названием

```
float absf(float fArgument); // функция для работы с float типами
int abs(int iArgument); // функция для работы с целочисленными параметрами
```

- Типичный представитель C интерфейса math.h и OpenGL, Vulkan
- Количество единиц интерфейса разрастается,
- Растет риск ошибки применения функции неверного типа...

Новшество C++ - переопределение функций (overload)

- Функции с разным набором параметров могут иметь одинаковое имя,
- Компилятор сам выберет какую функцию вызвать исходя из набора параметров,

```
float abs(float argument); // функция для работы с float типами
int abs(int argument); // функция для работы с целочисленными параметрами
double abs(float argument); // Егггг, необходимо переопределить параметр
```

- Если компилятор не сможет выбрать функцию, он выдаст ошибку о неоднозначности (ambiguity)
- Нельзя переопределить только возвращаемое значение.

Идентичность

Overrides

- В рамках большого продукта иногда возникает необходимость объявить несколько функций с одинаковой сигнатурой.
- Несколько внутренних модулей написанных независимыми разработчиками могут возникнуть одинаковые функции.
- Функция может быть воспринята компилятором как частичная спецификация существующего шаблона со схожей сигнатурой. Код будет работать совсем неожиданно.

External code

- Внешние - сторонние библиотеки (или 3rd-party) могут содержать функции с сигнатурами идентичными вашим.
- Защита внешних библиотек часто включает в себя добавление префикса к имени функции чтобы избежать дублирования: `sslOpen`, `glDraw`, `glutDraw`.
 - Префиксы могут быть длинными (`glaux`, `glut...`).
 - Интерфейс библиотеки становится громоздким.
 - Никто не гарантирует соблюдения этого правила всеми.



Вопрос

Как отличить свой код от внешнего?

namespace's

Copied City

Namespace

```
namespace MySpace    // имя неймспейса (оно же его идентификатор)
{
    int variable = 0; // содержимое неймспейса
}
```

- Разделитель областей видимости для кода,
- Все что объявлено внутри неймспейса должно быть определено внутри него же,
- Доступ извне к объектам неймспейса требует соблюдения следующих правил:
 - Использовать идентификатор неймспейса перед каждым обращением к его объекту,

```
MySpace    :: variable = 10;
идентификатор :: объект (это может быть переменная, функция, класс, объект или другой namespace)
```

- Использование спецификатора using namespace,
 - Using следует использовать только внутри CPP файлов, во избежании просачивания директивы в другие модули.

```
using namespace MySpace;
variable = 10; // переменная находится в namespace
```

Неймспейс не влияет на макро подстановки. Они остаются глобальными.



Copied Factory



Закрытая структура

- Структура которая объявляется в рамках одного блока (от { до }). Любая попытка добавить поля создав еще один блок с теми же параметрами воспринимается компилятором как ошибка, или другой объект с тем же именем будет создан (функции):
 - Функция,
 - Структура,
 - Класс.

Открытая структура

- Не ограничивается одним блоком. Можно создать второй блок с тем же именем и компилятор сгруппирует их в один объект:
 - Namespace,
 - Partial Class (C#).

```
// Исходный неймспейс
namespace MySpace
{
    int variable = 0;
}
```

```
// Его продолжение в другом файле
namespace MySpace
{
    int variable2 = variable + 1;
}
```

Многоуровневая защита

0 - глобальный неймспейс

- Все функции объявленные вне какого либо неймспейса на самом деле находятся внутри глобального неймспейса. Имени у него нет, а правило доступа к нему сохранилось:

```
::fopen("dignostics.log", "a+");  
//fopen - функция из глобального namespace, :: - опционален
```

1 - без вложений

2 и выше

- Добавление неймспейса в неймспейс тривиально, простое вложение,
- Чтобы продолжить заполнять вложенный неймспейс, нужно повторить весь путь до него.

```
namespace First           // имя корневого неймспейса  
{  
    namespace Second      // второй уровень  
    {  
        namespace Third   // третий уровень неймспейса  
        {  
            int internal1 = 0;  
        }  
    }  
};
```

```
namespace First::Second::Third // полный путь до него  
{  
    // продолжаем наш неймспейс  
    int internal2 = internal1 + 1;  
}
```



Доступ к данным

```
namespace MySpace { int variable = 0;}
```

Область видимости

- Только внутри блока, где using использовался:

```
{  
    using namespace MySpace;  
    variable = 10; // ok  
}  
variable = 5; // Error - вне области видимости
```

Небезопасен

- Приводит к ambiguity ошибке, если переменная объявлена и в глобальном namespace и в том которое мы используем (using),
- Если переменные объявлены на разных уровнях произойдет экранирование, работа будет вестись с переменной объявленной ближе всего к месту использования.

```
int variable = 2;  
int main(int argc, char** argv)  
{  
    using namespace MySpace;  
    variable = 10; // Error - две переменные одного уровня (глобальные) с одним именем
```

С функциями те же проблемы...

Всегда обращайтесь к элементам namespace через идентификатор namespace и ::



Вопрос

Библиотека примитивов **STL** (не опять а снова):

```
std::vector<uint32> myVector;
```

Вся библиотека STL расположена внутри namespace std.

Командный центр

Основной элемент библиотеки STL

- Большой набор оптимизированных контейнеров на все случаи,
- Удобство использования - единый интерфейс,
- Синтаксические конструкции для работы с контейнерами,

```
std::array<int, 256> staticArray;      // статический массив целых длиной 256 элементов
std::vector<int> iVector;              // динамический массив целых
std::list<double> list;                // двусвязный список из чисел с плавающей точкой
std::set<std::string> set;             // множество строк
std::map<std::string, int> dictionary; // словарь, поиск числа по строке
```

- Итд. итп.

Общие функции контейнеров (интерфейс)

- **size/empty** - получение размера контейнера (количества элементов), проверка на заполненность,
- **push/push_back/push_front/insert** - добавление элемента с автоматическим расширением контейнера,
- **pop/erase** - удаление элемента,
- **front/back** - получение значений в контейнере в первой и последней позиции,
- **begin/end** - получение итератора начала или конца контейнера.

На самом деле функции не такие уж и общие, их наличие зависит от того допускает ли структура данных те или иные операции.



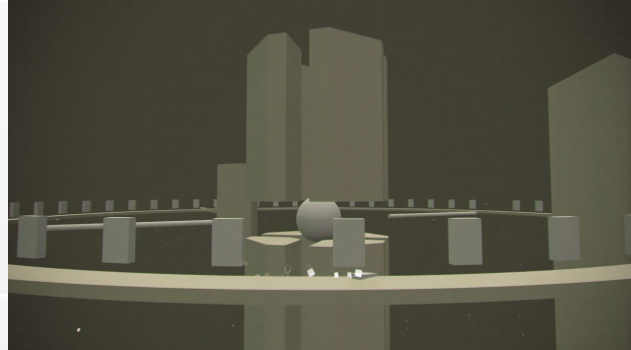
Внутренний сервер

Итераторы: `container<type>::iterator`

- Общие для всех контейнеров,
- Обобщенные функции STL не относящиеся к конкретным контейнерам умеют работать с итераторами:
 - Универсальность в ущерб производительности, часто используют алгоритмы “в лоб”.
- Используются для последовательного доступа к элементам контейнера:
 - Можно инкрементировать - для получения следующего элемента (++),
 - Можно декрементировать - для получения предыдущего элемента (--).
- Доступ к элементу контейнера по итератору осуществляется через разыменование (*iterator).
- Экзотические итераторы:
 - `Const_iterator` - то же самое, через него нельзя менять содержимое контейнера,
 - `rbegin/rend` - функции возвращающие `reverse_iterator` он движется от конца в начало (инкремент будет вести ближе к началу контейнера).

```
std::vector<uint32> myVector(100);
for (std::vector<uint32_t>::iterator it = myVector.begin(); it != myVector.end(); ++it) // можно auto
{
    *it = 10; // запись значения в текущий элемент контейнера
}
```

Итератор `end()` не содержит значений, его разыменование приведет к падению программы.



Кольцевой коридор



Циклы по содержимому контейнеров

- Цикл по итераторам - подходит для работы со структурой контейнера - добавить, удалить элемент...

```
std::vector<uint32> myVector(100);  
for (std::vector<uint32_t>::iterator it = myVector.begin(); it != myVector.end(); ++it)  
for (auto it = myVector.begin(); it != myVector.end(); ++it) // лучше, но ненамного
```

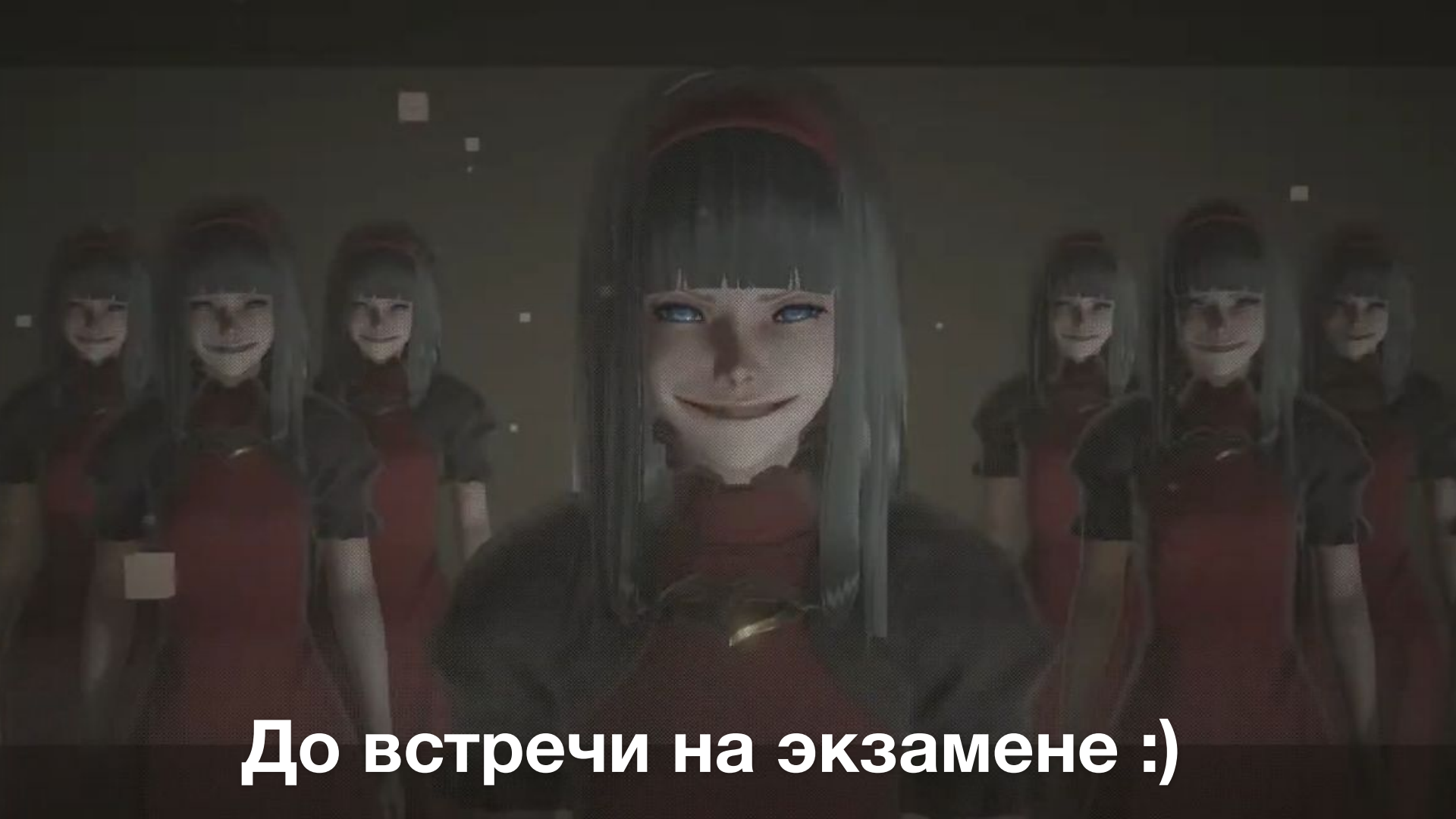
- for_each - is dead - туда ему и дорога...
- Modern foreach

```
for (const auto v : myVector) // readonly для простых типов  
for (const auto& v : myVector) // readonly для сложных типов  
for (auto& v : myVector)      // read/write  
{  
    v = 10;                  // запись значения в текущий элемент контейнера  
}
```

- Можно использовать для своего типа, требуется имитация итераторов (begin/end, и тип iterator)

Работает для ВСЕХ типов контейнеров

for (auto v : myVector) - источник большого числа ошибок, в цикле происходит заполнение, а по факту его нет...
поэтому проще всегда использовать ссылку, **for (auto& v : myVector)** или **for (const auto& v : myVector)**



До встречи на экзамене :)